

# Gen AI Academy

APAC Edition

Build in APAC. Build for the world.



## Participant Details

- a. Participant Name: Yasmeen Begum
- b. Problem Statement: Multi-Agent Productivity Assistant for Health & Wellness

## Brief about the idea

The idea is to build a **multi-agent productivity assistant** that focuses on health and wellness by coordinating tasks, schedules, and information. Using **Google ADK**, you define agents and workflows that can handle prompts, query data, and synthesize responses in a modular way. The **root agent** captures user input, passes it to a workflow agent, which then orchestrates sub-agents like a researcher and a formatter. Through **MCP integration**, the system connects to external tools such as calendars, task managers, and notes, enabling multi-step workflows like logging meals and scheduling workouts. **AlloyDB** serves as the structured data store, holding wellness plans, workouts, diet logs, appointments, and notes with relational integrity. Agents query AlloyDB via SQLAlchemy ORM tools to fetch personalized data. External knowledge sources like Wikipedia are combined with internal data to enrich responses. The assistant is deployed as an **API-based system** using FastAPI, containerized with Docker, and run on **Cloud Run** for scalability. This architecture demonstrates how agents, tools, and databases coordinate seamlessly to deliver real-world wellness workflows.

## Your solution should be able to explain the following:

- How did you approach the common problem statement and translate it into a working solution using ADK, MCP/AlloyDB AI while ensuring Hackathon's core requirements are met?
  - What real-world problem does your solution address, and what practical impact does it create for users?
  - What is the core approach or workflow behind your solution?
- 
- I have solved the problem by building a **multi-agent system** with Google ADK, which defines modular agents and workflows to handle user prompts.
  - Through **MCP integration**, the agents connect to external tools like calendars, tasks, and notes, enabling multi-step productivity workflows.
  - **AlloyDB** acts as the structured data store, holding wellness plans, workouts, diet logs, appointments, and notes for personalized insights.
  - The real-world problem addressed is **fragmented wellness management**, and our assistant unifies everything into one intelligent interface.
  - The core workflow is: root agent captures the prompt → researcher agent queries AlloyDB and external sources → formatter agent synthesizes results → API delivers a friendly response.

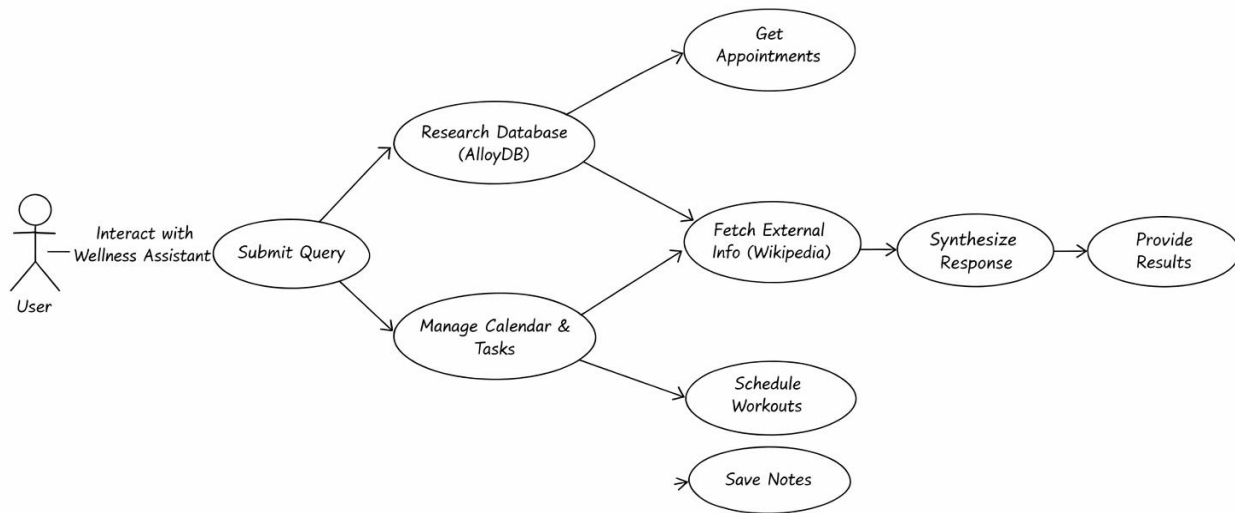
## Opportunities

- How different is it from any of the other existing ideas?
  - USP of the proposed solution
- 
- Our solution is unique because it combines **multi-agent orchestration, structured data, and external knowledge** in one workflow.
  - Most wellness apps only track workouts or meals separately, but we unify everything into a single intelligent assistant.
  - We use **Google ADK** to coordinate specialized agents, **MCP** to integrate calendars, tasks, and notes, and **AlloyDB** to store/query personalized wellness data.
  - The USP is that it doesn't just show raw data — it **synthesizes appointments, diet logs, workouts, and external health knowledge** into one conversational response.
  - This creates a practical impact: users no longer juggle multiple apps, they get a **holistic assistant** that manages wellness routines end-to-end.

## List of features offered by the solution

- **Multi-agent orchestration** using Google ADK to define modular agents and workflows.
- **Integration with external tools** via MCP (calendars, tasks, notes) for multi-step productivity workflows.
- **Structured data management** with AlloyDB storing wellness plans, workouts, diet logs, appointments, and notes.
- **Personalized insights** by combining user-specific AlloyDB data with external knowledge sources like Wikipedia.
- **Unified interface** that eliminates fragmentation across multiple wellness apps.
- **Conversational responses** synthesized by the formatter agent for user-friendly outputs.
- **Scalable deployment** using FastAPI, Docker, Artifact Registry, and Cloud Run.
- **Extensible design** to add new agents for other domains (finance, productivity, supply chain).
- **Automated workflows** such as logging meals, scheduling workouts, and tracking appointments.
- **Holistic wellness management** that covers physical health, diet, scheduling, and journaling in one assistant.

## Process flow diagram or Use-case diagram



## Wireframes/Mock diagrams of the proposed solution

← → ↺

https://lab2-cr-service-753573680490.us-central1.run.app/dev-ui

Agent Development Kit

wellness\_agent

wellness\_agent

Session ID: 4f5c4ddd-789e-497f-856fd-c4ceb04fedf7 Token Streaming

+ New Session

Delete

Download

🗑️

Trace

Trace | Events | State | Artifacts | Sessions | Eval

10:15:05 AM Root Agent: Received Show my appointments for today."

10:15:04 AM Workflow Agent: Initiating Sub-Agents

10:15:08 AM Researcher Agent: Querying AlloyDB (Appointments)

10:15:10 AM Researcher Agent: Fetthng Data from Wikipedia

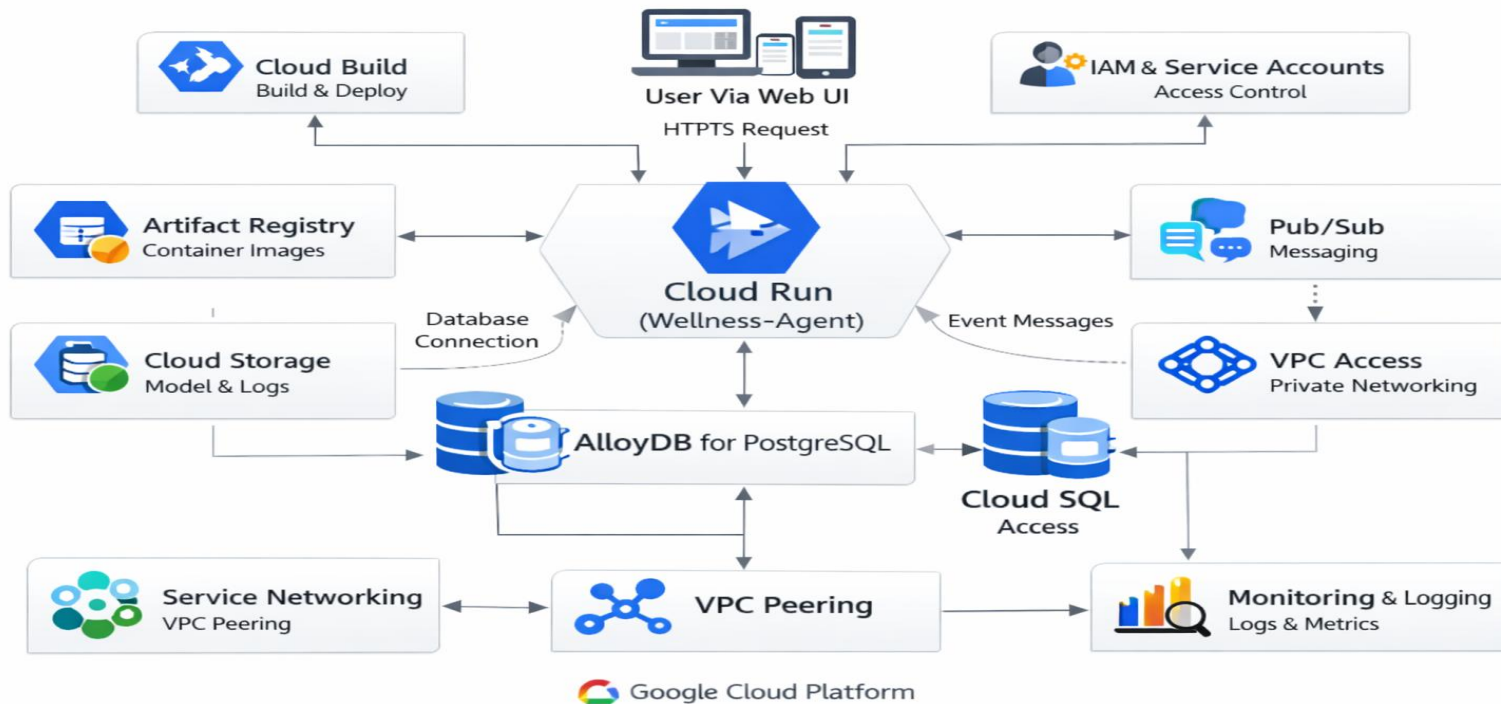
10:15:13 AM Formatter Agent: Synthesizing Response

10:15:16 AM Response Sent to User

Type a message

📎 ... 🎤 🎥

## Architecture diagram of the proposed solution:





## Technologies/Google Services used in the solution

(Why did you choose your specific AI stack and system design, and how does it support scalability and real-world deployment?)

- **Google ADK (Agent Development Kit):** Used to design modular, reusable agents and workflows. It ensures flexibility and easy scaling by allowing new agents to be added without changing the core logic.
- **MCP (Multi-Channel Protocol):** Enables seamless integration with external tools like calendars, tasks, and notes, supporting real-time coordination across multiple systems.
- **AlloyDB (Managed PostgreSQL):** Provides a secure, high-performance database for storing structured wellness data such as workouts, diet logs, and appointments.
- **FastAPI + Docker:** Used to build and containerize the API layer for fast, lightweight deployment.
- **Cloud Run + Artifact Registry:** Offers serverless, scalable deployment with automatic load balancing and version control.
- **Cloud IAM & Secret Manager:** Manage access and credentials securely for production environments.

-Why This Stack

- **Scalability:** Cloud Run auto-scales based on traffic; ADK workflows are modular and reusable.
  - **Reliability:** AlloyDB ensures consistent, high-speed data access for multiple agents.
  - **Security:** IAM and Secret Manager protect sensitive data and credentials.
  - **Real-world readiness:** The system can handle concurrent user requests, integrate external APIs, and adapt to new domains (finance, productivity, healthcare).
  - **Ease of deployment:** Docker and Artifact Registry make builds reproducible and portable across environments.
- This combination of **ADK, MCP, and AlloyDB** forms a robust, cloud-native AI stack that's scalable, secure, and ready for real-world use.

## Snapshots of the prototype

The screenshot displays the Google Cloud AlloyDB console for a project named 'My Project 30236'. The main view is the 'Overview' page for the 'my-prod-cluster'. The cluster is in a 'Ready' status. Key details include: Cluster ID 'my-prod-cluster', Version 'PostgreSQL 17' with an 'Upgrade' link, Cluster type 'Primary cluster', High availability 'Highly available (multi-zone)', Location 'us-central1 (Iowa)' with a 'Low CO2' indicator, Total storage used '71.01 MB', Network 'easy-alloydb-vpc', and Maintenance 'Upcoming: No maintenance currently scheduled. Edit'. A 'Show details' link is available for maintenance. The 'Data replication' section shows 'No streams found' and a 'Start stream' button. On the right, the 'Health issues' section lists a 'Primary cluster' issue: 'Not multi-region for disaster recovery', with a 'View issue details' link. The bottom of the screen shows a 'CLOUD SHELL Terminal' window with the session name '(able-bazaar-492510-c8)' and a taskbar with various application icons and system information (10:32 PM, 08-04-2026).

my-prod-cluster - Overview

console.cloud.google.com/alloydb/locations/us-central1/clusters/my-prod-cluster/overview?cloudshell=t...

Google Cloud

My Project 30236

Search (/) for resources, docs, products and more

AlloyDB for PostgreSQL / Search

All clusters > Primary cluster my-prod-cluster: Overview

Import Export Migrate database + Create secondary cluster Delete cluster Learn

### Cluster details

|                    |                                                    |
|--------------------|----------------------------------------------------|
| Status             | Ready                                              |
| Cluster ID         | my-prod-cluster                                    |
| Version            | PostgreSQL 17 Upgrade                              |
| Cluster type       | Primary cluster                                    |
| High availability  | Highly available (multi-zone)                      |
| Location           | us-central1 (Iowa) Low CO2                         |
| Total storage used | 71.01 MB                                           |
| Network            | easy-alloydb-vpc                                   |
| Maintenance        | Upcoming: No maintenance currently scheduled. Edit |

Show details

### Data replication

No streams found Start stream

### Health issues

Primary cluster

- Not multi-region for disaster recovery

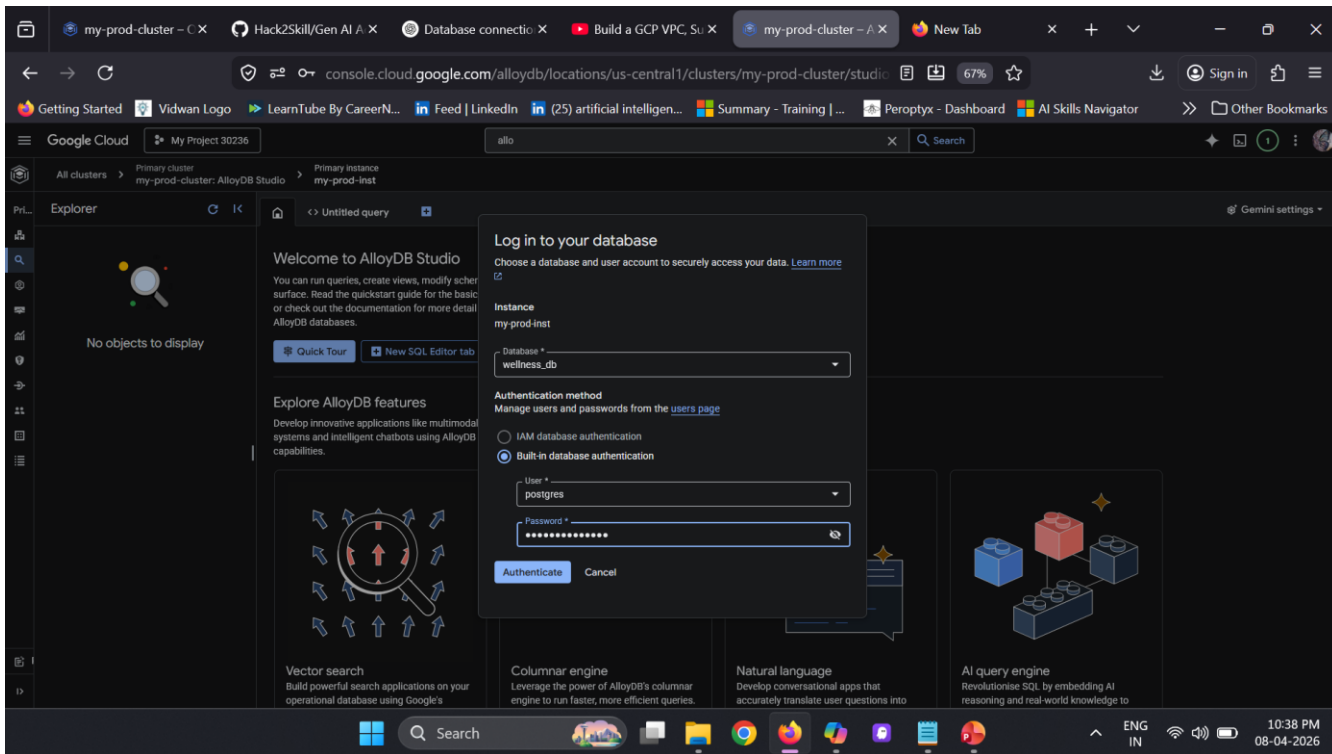
View issue details

CLOUD SHELL Terminal (able-bazaar-492510-c8)

Open editor

10:32 PM 08-04-2026

## Snapshots of the prototype



# Snapshots of the prototype

The screenshot displays the Google Cloud AlloyDB Studio interface. The left sidebar shows the Explorer view with a tree structure of the database schema, including tables like appointments, diet\_logs, wellness\_plans, and workouts. The main area is the SQL editor, which contains a query to create several tables and their relationships. A red error message at the bottom of the editor indicates a syntax error: "Syntax error at or near 'ON'". The query is as follows:

```
1 CREATE TABLE wellness_plans (  
2   plan_id SERIAL PRIMARY KEY,  
3   user_id VARCHAR(50) NOT NULL,  
4   plan_name VARCHAR(255),  
5   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
6 );  
7  
8 CREATE TABLE workouts (  
9   workout_id SERIAL PRIMARY KEY,  
10  plan_id INT REFERENCES wellness_plans(plan_id) ON DELETE CASCADE,  
11  title VARCHAR(255),  
12  recurrence_rule TEXT,  
13  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
14 );  
15  
16 CREATE TABLE diet_logs (  
17  diet_id SERIAL PRIMARY KEY,  
18  plan_id INT REFERENCES wellness_plans(plan_id) ON DELETE CASCADE,  
19  meal_date DATE NOT NULL,  
20  meal_description TEXT,  
21  calories INT,  
22  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
23 );  
24  
25 CREATE TABLE appointments (  
26  appointment_id SERIAL PRIMARY KEY,  
27  plan_id INT REFERENCES wellness_plans(plan_id) ON DELETE CASCADE,  
28  title VARCHAR(255),  
29  appointment_date TIMESTAMP NOT NULL,  
30  reminder_minutes_before INT,  
31  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
32 );  
33  
34 CREATE TABLE notes (  
35  note_id SERIAL PRIMARY KEY,  
36  plan_id INT REFERENCES wellness_plans(plan_id) ON DELETE CASCADE,  
37  note_title VARCHAR(255)
```

The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 10:42 PM on 08-04-2026.

# Snapshots of the prototype

The screenshot displays the Google Cloud AlloyDB Studio interface. The top navigation bar shows the project 'my-prod-cluster - AlloyDB Studio' and the instance 'my-prod-inst'. The left sidebar contains a tree view of the database schema, including tables like 'appointments', 'diet\_logs', 'notes', 'wellness\_plans', and 'workouts'. The main area shows a SQL query being executed, with the results displayed below. The query is a series of INSERT statements into various tables, including 'wellness\_plans', 'workouts', 'diet\_logs', and 'appointments'. The execution time is 25.8 ms, and the status is 'Valid'.

```
-- wellness_plans (6 users)
2 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('neha', 'Weekly Health Routine');
3 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('arjun', 'Fitness Boost Plan');
4 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('sara', 'Balanced Lifestyle');
5 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('mike', 'Weight Loss Journey');
6 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('nisha', 'Yoga & Mindfulness');
7 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('rahul', 'Strength Training Plan');
8 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('anika', 'Healthy Living Routine');
9 INSERT INTO wellness_plans (user_id, plan_name) VALUES ('11', 'Cardio & Endurance');

-- Workouts (8 entries)
12 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (1, 'Yoga Session', 'RRULE:FREQ=DAILY;BYDAY=MO,WE');
13 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (2, 'Yoga Class', 'RRULE:FREQ=DAILY;BYDAY=TU,TH');
14 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (3, 'Morning Run', 'RRULE:FREQ=DAILY;INTERVAL=1');
15 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (4, 'Cycling', 'RRULE:FREQ=DAILY;BYDAY=SA');
16 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (5, 'Meditation', 'RRULE:FREQ=DAILY;INTERVAL=1');
17 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (6, 'Strength Training', 'RRULE:FREQ=DAILY;BYDAY=MO,FR');
18 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (7, 'Swimming', 'RRULE:FREQ=DAILY;BYDAY=SU');
19 INSERT INTO workouts (plan_id, title, recurrence_rule) VALUES (8, 'HIIT Workout', 'RRULE:FREQ=DAILY;BYDAY=TU,TH');

-- Diet Logs (8 entries)
22 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (1, '2026-04-07', 'Grilled chicken salad', 450);
23 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (2, '2026-04-07', 'Vegetable stir fry', 380);
24 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (3, '2026-04-07', 'Oatmeal with fruits', 300);
25 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (4, '2026-04-07', 'Protein shake', 250);
26 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (5, '2026-04-08', 'Green smoothie', 200);
27 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (6, '2026-04-08', 'Chicken curry with rice', 680);
28 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (7, '2026-04-08', 'Avocado toast', 350);
29 INSERT INTO diet_logs (plan_id, meal_date, meal_description, calories) VALUES (8, '2026-04-08', 'Salmon with quinoa', 500);

-- Appointments (8 entries)
32 INSERT INTO appointments (plan_id, title, appointment_date, reminder_minutes_before) VALUES (1, 'Doctor Appointment', '2026-04-18 10:00:00', 30);
```

Execution time: 25.8 ms

Statement executed successfully

## Snapshots of the prototype

The screenshot displays the Agent Development Kit (ADK) interface within a web browser. The browser's address bar shows the URL: `lab2-cr-service-753573680490.us-central1.run.app/dev-ui/?app=wellness_agent&session=5aa74905-f94f-4b4d-c42b71e1af54`. The interface is divided into several sections:

- Header:** Includes the "Agent Development Kit" logo and a session ID: `5aa74905-f94f-4b4d-b85e-c42b71e1af54`. There are also controls for "Token Streaming" and a "New Session" button.
- Left Sidebar:** Features a dropdown menu set to "wellness\_agent" and tabs for "Trace", "Events", "State", "Artifacts", "Sessions", and "Eval". Below these is a section labeled "Invocations".
- Main Chat Area:** Contains a blue button with the text "show me upcoming appointments and workouts" and a user profile icon. Below this is a placeholder for a response, represented by a robot icon and a series of dots. At the bottom, there is a text input field labeled "Type a Message..." with icons for attachments, a menu, voice recording, and video recording.

The Windows taskbar at the bottom shows the time as 11:41 PM on 08-04-2026, with the language set to ENG IN.

Google Cloud 

# Gen AI Academy

APAC Edition

# Thank you

Build in APAC. Build for the world.

